İZMİR UNIVERSITY OF ECONOMICS

SE 322 - SE 318
SOFTWARE VERIFICATION AND VALIDATION
SPRING 2024-2025

INCIDENT MANAGEMENT SYSTEM

ÖYKÜ IREM OPLAZGIL

MEHMET TAŞOĞLU

BERKAY IŞIK

ERAY ÖZKAN

MUHAMMED SAIT DOKUR

ÇAĞAN PARLAPAN

UNIT TEST DOCUMENT

Version <*3.0*>

<*22.05.2025*>.

VERSION HISTORY

VERSION 1.0 (26.03.2025)
There are 5 High Priority Functional Requirements in this version (V1) High FR-1) Users should be able to log in to the system with predefined ID information. High FR-2) Users should be able to create a new incident report. High FR-3) Users should be able to type the incident description/definition. High FR-4) Users should be able to select the status of the incident (Started, In Progress, Done, Delayed). High FR-5)Users should be able to select the priority of the incident (Low, Medium, Important, Urgent). All of these requirements constitute the basic functionality of the system and have been completed during the coding process. Thus, it has become possible for users to log in to the system securely, define new events, and systematically enter descriptions, statuses, and priority information about these events.
VERSION 2.0 (31.04.2025)
There are 3 Moderate Priority Functional Requirements in this version (V2) Moderate FR-1) Users should be able to select the start and end date of the incident if necessary. Moderate FR-2) Users can edit/delete the existing incident report if necessary. Moderate FR-3) Users should be able to upload documents to the incident report if necessary. All of these requirements support the functionality of the system and have been implemented and tested during the coding process; thus, users can select the start and end dates of the incident when necessary, edit and delete existing incident reports, and upload files to the system when they need relevant documents, thus making the incident management process more flexible and comprehensive.
VERSION 3.0 (20.05.2025)

There are 2 Low Priority Functional Requirements in this version (V3)

Low FR-1) Users can list incidents on that date by typing a date.

Low FR-2) Users can see/review previous incident posts.

These requirements are complementary functions aimed at improving the user experience of the system and were implemented during the coding process so that users can list events on a specific date by typing in a date and view and examine event reports created in the past, which made the system more usable and effective in terms of accessing information and tracking records.

INTRODUCTION

1.1 PURPOSE OF THE TEST CASE DOCUMENT

The primary purpose of the this Test Case Document is to ensure the quality, functionality, and reliability of the software testing. All of case is designed to verify whether the software components work as expected according to the defined functional requirements. This process helps identify possible bugs, inconsistencies, or security vulnerabilities early in the development cycle. In addition it includes regression testing to verify that previously working features continue to work correctly after system updates or changes.

Beyond technical validation, this document serves as a communication tool between developers, testers, project managers, and users. It promotes alignment among all parties by providing a clear overview of what is being tested, the progress of testing activities, and the results generated. This improves collaboration, helps manage expectations, and ensures a consistent understanding of the system's behavior under different conditions.

For this project specifically, the Test Case Document focuses on verifying the functionality of the Incident Management System. This includes ensuring that users can log in to the system, generate incident reports, add title and describe incidents in detail, then define their status and priority. It also includes more features such as setting start and end dates, editing or deleting previous reports, uploading documentation, filtering incidents by date, and reviewing all incident records. All of these capabilities are critical to enabling a consistent, user-friendly, and functional system that supports efficient incident tracking and management.

1.2 CONSTRAINTS

The test cases documented in this report were executed under a defined set of technical and environmental constraints. The application under test was developed using Java and the testing process was conducted within the constraints of the selected development framework and tools.

Unit and integration tests were implemented using JUnit, the standard testing framework for Java-based applications. Manual functional tests were also performed to validate user interactions and UI-based requirements that could not be automated within the current scope.

2 UNIT TEST FRAMEWORK: *<JUNIT4>*

JUnit may be a well-known open-source testing framework for Java and is primarily used to write and run unit tests. It uses annotations to characterize unit testing strategies, setup and unload operations, and other metadata within test classes. Comments like @Test , @Before @After , and @Ignore solve the test writing method and make code reviewing easier. JUnit offers a wide set of assertion strategies to verify expected results in tests. Engineers can use strategies like "assertEquals", "assertFalse", "assertTrue", and more to verify conditions and behaviors in their code. JUnit supports expected exception maintenance in test cases. JUnit supports parameterized testing, which allows engineers to test the same utility with different input values. Parameterized testing increases test coverage and reduces code duplication by allowing different test cases to be executed with varying parameters. Overall, JUnit offers a powerful and adaptable framework to create and run unit tests in Java initiatives.

3 TEST CASES

Test Case 1	
Test Definition	
Test if FileManager.createDocumentsDirectory() creates the expected directory.	
Input Value	
Input Value: N/A (method call without parameters).	
Expected Value	**Actual Value**
A directory named "incident_documents/" should exist	The directory "incident_documents/" exists if the method was successful
Result of Test Case	*Successful*
Test Script	

```
public class CreateDirectoryTest {
   private FileManager fileManager;
   @Before
   public void setUp() {
      fileManager = new FileManager();

fileManager.createDocumentsDirectory();
   }
   @Test
   public void
testCreateDocumentsDirectory_Positive()
{   assertTrue("Documents directory
should exist", new
File("incident_documents/").exists());
   }
   @Test
   public void
testCreateDocumentsDirectory_Negative()
{
      File dir = new
File("incident_documents_wrong");
      assertFalse("Wrong directory should
not exist", dir.exists());
   }
}
```

Test Case 2

Test Definition	
Test the addition and removal of attachments to an incident, including handling of duplicate file paths and attempts to remove non-existent attachments.	
Input Value	
Test scenarios include adding the same file path twice, attempting to remove a file path that was not added, adding three attachments and removing one, and adding two different file paths.	
Expected Value	**Actual Value**
Verify the attachment list in different scenarios: containing both paths, remaining unchanged with 1 item, allowing duplicates with 2 identical items, and containing 2 items after one has been removed.	Attachment list contains 1 item, 2 item and both paths.
Result of Test Case	*Successful*
Test Script	

```
public class IncidentAttachmentTest {
   private Incident incident;
   @Before
   public void setUp() {
      incident = new Incident("INC007", "user1",
"Attachment Test",
                LocalDate.now(),
LocalDate.now().plusDays(7),
                "Testing attachment functionality",
"Started", "Low");
   }@Test
   public void testPositiveAttachmentAddition() {
      List<String> attachments =
incident.getAttachments();
      assertTrue("Fresh incident should have no
attachments", attachments.isEmpty());
      incident.addAttachment("path/to/document1.pdf");
     incident.addAttachment("path/to/document2.docx");
      attachments = incident.getAttachments();
      assertEquals("Should have 2 attachments", 2,
attachments.size());
      assertTrue("Should contain first attachment",
attachments.contains("path/to/document1.pdf"));
      assertTrue("Should contain second attachment",
attachments.contains("path/to/document2.docx"));
   } @Test
   public void testPositiveAttachmentRemoval() {
```

```
        incident.addAttachment("path/to/document1.pdf");
      incident.addAttachment("path/to/document2.docx");
        incident.addAttachment("path/to/document3.txt");
incident.removeAttachment("path/to/document2.docx");
        List<String> attachments =
incident.getAttachments();
        assertEquals("Should have 2 attachments after
removal", 2, attachments.size());
        assertTrue("Should still contain first attachment",
attachments.contains("path/to/document1.pdf"));
        assertFalse("Should not contain removed
attachment",
attachments.contains("path/to/document2.docx"));
        assertTrue("Should still contain third attachment",
attachments.contains("path/to/document3.txt"));
    } @Test
    public void testNegativeAttachmentRemoval() {
        incident.addAttachment("path/to/document1.pdf");
incident.removeAttachment("path/to/nonexistent.txt");
        List<String> attachments =
incident.getAttachments();
        assertEquals("Should still have 1 attachment", 1,
attachments.size());
        assertTrue("Should still contain original
attachment",
attachments.contains("path/to/document1.pdf"));
    }@Test
    public void testDuplicateAttachments() {
        incident.addAttachment("path/to/document1.pdf");
        incident.addAttachment("path/to/document1.pdf");
        List<String> attachments =
incident.getAttachments();
        assertEquals("Should have 2 attachments
(ArrayList allows duplicates)", 2, attachments.size());
    }
}
```

Test Case 3
Test Definition
Test if incident dates are correctly stored and updated.
Input Value

Initial dates (May 1-15, 2025) and updated dates (June 1-30, 2025).	
Expected Value	**Actual Value**
Initial dates should match input, then dates should update correctly.	Dates match initial input and are properly updated when changed.
Result of Test Case	***Successful***
Test Script	

```
public class IncidentDatesTest {
   @Test
   public void testPositiveDateHandling() {
      LocalDate startDate = LocalDate.of(2025, 5, 1);
      LocalDate endDate = LocalDate.of(2025, 5, 15);
      Incident incident = new Incident("INC003", "user1",
"Date Test",
            startDate, endDate, "Testing date handling",
"Started", "Low");
      assertEquals("Start date should match the input",
startDate, incident.getStartDate());
      assertEquals("End date should match the input",
endDate, incident.getEndDate());
      LocalDate newStartDate = LocalDate.of(2025, 6, 1);
      LocalDate newEndDate = LocalDate.of(2025, 6, 30);
      incident.setStartDate(newStartDate);
      incident.setEndDate(newEndDate);
      assertEquals("Start date should be updated",
newStartDate, incident.getStartDate());
      assertEquals("End date should be updated",
newEndDate, incident.getEndDate());
   }
```

Test Case 4
Test Definition
Test updating the incident description with valid text and with an empty string.
Input Value
1)Updated description with more details about the incident. 2) ""

Expected Value	Actual Value
Description matches the new input and IllegalArgumentException thrown	Description matches the new input and IllegalArgumentException thrown
Result of Test Case	***Successful***
Test Script	

```
public class IncidentDescriptionTest {
   private Incident incident;
   private final LocalDate startDate =
LocalDate.now();
   private final LocalDate endDate =
startDate.plusDays(7);
   @Before
   public void setUp() {
      incident = new Incident("INC001",
"testuser", "Test Incident",
            startDate, endDate, "Initial
description", "Started", "Medium");
   }
@Test
   public void testUpdateDescriptionValid() {
      String newDescription = "Updated
description with more details about the
incident";
incident.setDescription(newDescription);
      assertEquals(newDescription,
incident.getDescription());
   }
@Test(expected =
IllegalArgumentException.class)
   public void testEmptyDescription() {
      String emptyDesc = "";
     incident.setDescription(emptyDesc);
   }
```

Test Case 5
Test Definition
Test whether saving an incident creates the correct file and does not generate any incorrect files.
Input Value
Incident with user ID "user1".

Expected Value	Actual Value
1)File "user1_incident.txt" exists 2) File "incorrect_file.txt" does not exist	1)File "user1_incident.txt" exists 2) File "incorrect_file.txt" does not exist
Result of Test Case	***Successful***
Test Script	

```
public class IncidentFileTest {
   private FileManager fileManager;
   private Incident testIncident;
   @Before
   public void setUp() {
      fileManager = new FileManager();
      testIncident = new Incident("INC001",
"user1", "Test Incident", LocalDate.now(),
LocalDate.now().plusDays(7),
"Test Description", "Started", "High");
   }@Test
   public void
testSaveIncidentToFile_Positive() {
fileManager.saveIncidentToFile(testIncident);
      assertTrue("Incident file should be
created", new
File("user1_incident.txt").exists());
   }@Test
   public voidtestSave
IncidentToFile_Negative() {
fileManager.saveIncidentToFile(testIncident);
      assertFalse("Incorrect file should not be
created", new
File("incorrect_file.txt").exists());
   }
}
```

Test Case 6
Test Definition
Test, changing an incident's priority using valid options and handling of non-standard priority values
Input Value

1) "Medium", "Important", "Urgent" 2) "Critical", "" (empty string)	
Expected Value	**Actual Value**
1) Priority updates to match each new value 2)System accepts any string as priority	1) Priority updates to match each new value 2)System accepts any string as priority
Result of Test Case	***Successful***
Test Script	

```
public class IncidentPriorityTest {
   private Incident incident;
   @Before
   public void setUp()  incident = new
Incident("INC002", "user1", "Priority
Test",
LocalDate.now(),
LocalDate.now().plusDays(7),
"Testing priority changes", "Started",
"Low");
   }@Test
   public void
testPositivePriorityChanges() {
      assertEquals("Initial priority
should be 'Low'", "Low",
incident.getPriority());
      incident.setPriority("Medium");
      assertEquals("Priority should be
updated to 'Medium'", "Medium",
incident.getPriority());
      incident.setPriority("Important");
      assertEquals("Priority should be
updated to 'Important'", "Important",
incident.getPriority());
      incident.setPriority("Urgent");
      assertEquals("Priority should be
updated to 'Urgent'", "Urgent",
incident.getPriority());
   }
   // negative
   @Test
   public void
testNegativePriorityChanges() {
      incident.setPriority("Critical");
```

```
        assertEquals("System should
accept any priority string", "Critical",
incident.getPriority());
        incident.setPriority("");
        assertEquals("System should
accept empty priority", "",
incident.getPriority());
        incident.setPriority("Low");
        assertEquals("Priority should be
reset to 'Low'", "Low",
incident.getPriority());
    }
}
```

Test Case 7

Test Definition

Verify that valid status values can be set and retrieved correctly from an Incident object, and test whether the class accepts non-standard status values without validation.

Input Value

1 Initial status: "Started"

Set status to In Progress/Done/Delayed.

2) Set status to "Invalid Status"

Set status to empty string ""

Reset status to "Started"

Expected Value	Actual Value
1) Initial status should be "Started" Status should be updated to In Progress/Done/Delayed 2) System should accept any status string, status becomes "Invalid Status" System should accept empty status, status becomes "" Status should be reset to "Started"	1) Initial status should be "Started" Status should be updated to In Progress/Done/Delayed 2) System should accept any status string, status becomes "Invalid Status" System should accept empty status, status becomes "" Status should be reset to "Started"
Result of Test Case	***Successful***

Test Script

```
public class IncidentStatusTest {
    private Incident incident;
    @Before
    public void setUp() {
```

```
        incident = new Incident("INC001",
"user1", "Status Test",
                LocalDate.now(),
LocalDate.now().plusDays(7),
                "Testing status changes",
"Started", "Medium");
    }
    //  positive
    @Test
    public void
testPositiveStatusChanges() {
        assertEquals("Initial status
should be 'Started'", "Started",
incident.getStatus());
        incident.setStatus("In Progress");
        assertEquals("Status should be
updated to 'In Progress'", "In
Progress", incident.getStatus());
        incident.setStatus("Done");
        assertEquals("Status should be
updated to 'Done'", "Done",
incident.getStatus());
        incident.setStatus("Delayed");
        assertEquals("Status should be
updated to 'Delayed'", "Delayed",
incident.getStatus());
    }
    //  negative
    @Test
    public void
testNegativeStatusChanges() {
        incident.setStatus("Invalid
Status");
        assertEquals("System should
accept any status string", "Invalid
Status", incident.getStatus());
        incident.setStatus("");
        assertEquals("System should
accept empty status", "",
incident.getStatus());
        incident.setStatus("Started");
        assertEquals("Status should be
reset to 'Started'", "Started",
incident.getStatus());
```

```
    }
}
```

Test Case 8	
Test Definition	
Verifies that an Incident object can be created with a valid title and that the title value is correctly stored and retrieved.	
Input Value	
1)Create Incident with title "Valid Title" **2) Create Incident with title set to null**	
Expected Value	**Actual Value**
1) Title should match "Valid Title" **2) NullPointerException should be thrown**	**1)Title should match "Valid Title"** **2)NullPointerException should be thrown**
Result of Test Case	***Successful***
Test Script	

```
public class
IncidentTitleValidationTest {
   @Test
   public void testValidTitle_Positive()
{
  Incident incident = new
Incident("INC999", "userX", "Valid
Title", LocalDate.now(),
LocalDate.now().plusDays(5),
              "desc", "Started", "High");
      assertNotNull("Incident should
be created with valid title", incident);
      assertEquals("Title should
match", "Valid Title",
incident.getTitle());
   }
   @Test(expected =
NullPointerException.class)
   public void testNullTitle_Negative()
{
      new Incident("INC999", "userX",
null, LocalDate.now(), null, "desc",
"Started", "High");
```

```
    }
}
```

Test Case 9

Test Definition

Verifies that valid status values can be set and retrieved correctly from an Incident object, and tests whether non-standard status values are accepted without validation.

Input Value

1)user1
2)""(empty String)

Expected Value	Actual Value
1)User should be correctly assigned as "user1" 2)Empty user string should be preserved	1) User is assigned as "user1" 2) User is stored as empty string ""
Result of Test Case	***Successful***

Test Script

```
public class
IncidentUserAssignmentTest {
// positive
@Test
public void
testPositiveUserAssignment() {
Incident incident = new
Incident("INC008", "user1", "User
Assignment Test",
LocalDate.now(),
LocalDate.now().plusDays(7),
"Testing user assignment", "Started",
"Low");
assertEquals("User should be
correctly assigned", "user1",
incident.getUser());
} @Test
public void testEmptyUserHandling() {
Incident incident = new
Incident("INC011", "", "Empty User
Test",
LocalDate.now(),
LocalDate.now().plusDays(7),
```

```
"Testing with empty user", "Started",
"Low");
assertEquals("Empty user string
should be preserved", "",
incident.getUser());
}
```

Test Case 10	
Test Definition	
Verifies that the User class correctly returns the assigned user ID upon object creation and tests whether the authentication method properly rejects login attempts with an incorrect password.	
Input Value	
1)ID:testuser-password:password123 2) ID:testuser-password:wrongpassword	
Expected Value	**Actual Value**
1)getUserId() should return "testuser" 2)Fail (return false)	1)getUserId() should return "testuser" 2) 2)Fail (return false)
Result of Test Case	*Successful*
Test Script	

```
public class UserTest {
   //  positive
   @Test
   public void testGetUserId() {
      User user = new User("testuser",
"password123");
      assertEquals("Should return correct user
ID", "testuser", user.getUserId());
   } @Test
   public void
testFailedAuthenticationWithWrongPassword()
{
      User user = new User("testuser",
"password123");
```

```
        assertFalse("Should  reject  authentication
with wrong password",
                user.authenticate("testuser",
"wrongpassword"));
    }
```

Test Case 11	
Test Definition	
Verify that the method correctly returns true when receiving uppercase "Y" and "N" as input.	
Input Value	
1-"Y\n" 2- "N\n"	
Expected Value	**Actual Value**
1-return true 2-return false	**1-return true 2-return false**
Result of Test Case	***Successful***
Test Script	

```
public class askYesNoTest {
    private final InputStream originalIn =
System.in;
    @Before
    public void setUp() {
        InputUtility.resetScanner();
    }
    @After
    public void tearDown() {
        System.setIn(originalIn);
        InputUtility.resetScanner();
    }
    @Test
    public void
testAskYesNo_CorrectlyHandlesYInput() {
```

```
        ByteArrayInputStream in = new
ByteArrayInputStream("Y\n".getBytes());
        System.setIn(in);
        InputUtility.resetScanner();

        boolean result =
InputUtility.askYesNo("Do you want to
continue? (Y/N): ");
        assertTrue("Method should return true for
'Y' input", result);
    }
    @Test
    public void
testAskYesNo_CorrectlyHandlesNInput() {
        ByteArrayInputStream in = new
ByteArrayInputStream("N\n".getBytes());
        System.setIn(in);
        InputUtility.resetScanner();
        boolean result =
InputUtility.askYesNo("Do you want to
continue? (Y/N): ");
        assertFalse("Method should return false
for 'N' input", result);
    }
```

Test Case 12	
Test Definition	
Verify that the method correctly processes a valid document path input and displays appropriate prompts and Verify that the method handles invalid document paths appropriately and continues prompting for input.	
Input Value	
1- Input Stream: "valid/path/file.txt\n\n" 2- Input Stream: "invalid/path/file.txt\n\n"	
Expected Value	**Actual Value**
The method should prompt the user to enter a file path The output should contain the text "Enter the file path to attach"	The method should prompt the user to enter a file path The output should contain the text

	"Enter the file path to attach"
Result of Test Case	***Successful***
Test Script	

```
public class AttachDocumentsToIncidentTest {
   private final ByteArrayOutputStream outContent = new
ByteArrayOutputStream();
   private final PrintStream originalOut = System.out;
   private final InputStream originalIn = System.in;
   private IncidentManager incidentManager;
   private Incident testIncident;
   @Before
   public void setUp() {
      System.setOut(new PrintStream(outContent));
      incidentManager = new IncidentManager(new
HashMap<>(), "testuser");
      testIncident = new Incident("1", "testuser", "Test
Incident", LocalDate.now(), null, "Description", "Open",
"High");
      InputUtility.resetScanner();
   }
   @After
   public void tearDown() {
      System.setOut(originalOut);
      System.setIn(originalIn);
      InputUtility.resetScanner();
   @Test
   public void testAttachValidDocumentPath() {
      String fakeInput = "valid/path/file.txt\n\n";
      ByteArrayInputStream inputStream = new
ByteArrayInputStream(fakeInput.getBytes());
      System.setIn(inputStream);
      InputUtility.resetScanner();
incidentManager.attachDocumentsToIncident(testIncident);
      String output = outContent.toString();
      assertTrue(output.contains("Enter the file path to
attach"));
   }
   @Test
   public void testAttachInvalidDocumentPath() {
      String fakeInput = "invalid/path/file.txt\n\n";
```

```
        ByteArrayInputStream inputStream = new
ByteArrayInputStream(fakeInput.getBytes());
        System.setIn(inputStream);
        InputUtility.resetScanner();
incidentManager.attachDocumentsToIncident(testIncident);
        String output = outContent.toString();
        assertTrue(output.contains("Enter the file path to
attach"));
    }
```

Test Case 13	
Test Definition	
Verify that the method correctly attaches a document to an incident when provided with a valid file path. Verify that the method correctly handles the case when an invalid file path is provided.	
Input Value	
1-"test_doc.txt" 2-"nonexistent_file.txt"	
Expected Value	**Actual Value**
1-Return message containing "Document attached successfully. 2-Return message: "File does not exist. Please enter a valid file path."	1-Return message containing "Document attached successfully. 2-Return message: "File does not exist. Please enter a valid file path."
Result of Test Case	*Successful*
Test Script	

```
public class AttachDocumentTest {
    private static final String TEST_FILE_NAME =
"test_doc.txt";
    private static final String INVALID_FILE_NAME =
"nonexistent_file.txt";
    private Incident incident;
    @Before
    public void setUp() throws IOException {
        File file = new File(TEST_FILE_NAME);
        if (!file.exists()) {
            try (FileWriter writer = new FileWriter(file)) {
                writer.write("Test document content");
```

```
 } } incident = new
Incident(UUID.randomUUID().toString().substring(0, 8),
"testUser",
            "Test Title", null, null, "Test Description", "Open",
"Low");
      FileManager.createDocumentsDirectory();
   }@After
   public void tearDown() {
      File original = new File(TEST_FILE_NAME);
      if (original.exists()) original.delete();
      for (String attachment : incident.getAttachments()) {
         new File(attachment).delete(); }}
   @Test
   public void testAttachDocument_Positive() {
      String result = FileManager.attachDocument(incident,
TEST_FILE_NAME);
      assertTrue(result.contains("Document attached
successfully"));
      assertEquals(1, incident.getAttachments().size());
      File attachedFile = new
File(incident.getAttachments().get(0));
      assertTrue(attachedFile.exists());
   }@Test
   public void testAttachDocument_Negative_InvalidPath()
{ String result = FileManager.attachDocument(incident,
INVALID_FILE_NAME);
      assertEquals("File does not exist. Please enter a valid
file path.", result);
      assertTrue(incident.getAttachments().isEmpty());
   }
}
```

Test Case 14	
Test Definition	
Verify that the method correctly creates a new event with all valid input and does not add documents when the user declines and Verify that the method accepts and processes an incident with an empty title field.	
Input Value	
1- Title: "Test Incident" 2- Title: ""	
Expected Value	**Actual Value**

Output should contain "Incident Created and Saved!"	Output contains "v3.Incident Created and Saved!"
Result of Test Case	***Successful***
Test Script	

```
public class CreateIncidentTest {
   private final ByteArrayOutputStream outContent
= new ByteArrayOutputStream();
   private final PrintStream originalOut =
System.out;
   private final InputStream originalIn = System.in;
   private IncidentManager manager;
   private Map<String, List<Incident>>
userIncidents;
   @Before
   public void setUp() {
      System.setOut(new PrintStream(outContent));
      userIncidents = new HashMap<>();
      userIncidents.put("user1", new ArrayList<>());
      manager = new
IncidentManager(userIncidents, "user1");
   }
   @After
   public void tearDown() {
      System.setOut(originalOut);
      System.setIn(originalIn);
      outContent.reset();
   } @Test
   public void testCreateIncidentPositive() {
      String input = String.join("\n",
            "Test Incident",      // title
            "2024-05-01",         // start date
            "2024-05-10",         // end date
            "Test description",    // description
            "1",                  // status option
            "2",                  // priority option
            "N"                    // attach documents ) +
"\n";
      System.setIn(new
ByteArrayInputStream(input.getBytes()));
      InputUtility.resetScanner
```

```
assertEquals("Started",
created.getStatus());
   }
   @Test
   public void
testCreateIncidentWithEm
ptyTitle() {
      String input =
String.join("\n",
            "",                   //
title empty
            "2024-05-01",
// start date
            "2024-05-02",
// end date
            "Short
description",   //
description
            "1",                  //
status input
            "2",                  //
priority input
            "N"                    //
askYesNo
      ) + "\n";
      System.setIn(new
ByteArrayInputStream(inp
ut.getBytes()));

InputUtility.resetScanner()
;

manager.createIncident();
      String output =
outContent.toString();

assertTrue(output.contain
```

```
        manager.createIncident();
        String output = outContent.toString();
 assertTrue(output.contains("v3.Incident Created and Saved!"));
        assertEquals(1, userIncidents.get("user1").size());
        Incident created = userIncidents.get("user1").get(0);
        assertEquals("Test Incident", created.getTitle());
s("v3.Incident Created and Saved!"));
        List<Incident> incidents = userIncidents.get("user1");
        assertEquals(1, incidents.size());
        assertEquals("", incidents.get(0).getTitle());
    }
}
```

Test Case 15

Test Definition

Tests that the deleteIncidentAttachments method correctly deletes all attachment files associated with an incident.

Input Value

Incident object with ID "test-id", user "test-user", title "title", description "desc", status "status", priority "priority"

Expected Value	Actual Value
1)Both temporary files should no longer exist after callingFileManager.deleteIncidentAttachments(incident) 2) The attachment list should still contain the non-existent file path after calling FileManager.deleteIncidentAttachments(incident)	1)assertFalse(new File(path1).exists()) 2)assertTrue("Attachment list should still contain the attachment", incident.getAttachments().contains(nonExistentPath))

Result of Test Case *Successful*

Test Script

```
public class DeleteIncidentAttachmentsTest {
   private Incident incident;
   private List<String> tempFilePaths;
   @Before
   public void setUp() throws IOException {
       incident = new Incident("test-id", "test-user", "title", null, null, "desc", "status", "priority");
       tempFilePaths = new ArrayList<>();
}   @After
   public void tearDown() {
```

```
        for (String path : tempFilePaths) {
            try {
                Files.deleteIfExists(new
File(path).toPath());
            } catch (IOException ignored) {}
}}@Test
    public void
testDeleteIncidentAttachments_Positive() throws
IOException {
        File tempFile1 =
File.createTempFile("test_doc1_", ".txt");
        File tempFile2 =
File.createTempFile("test_doc2_",".txt");
        String path1 = tempFile1.getAbsolutePath();
        String path2 = tempFile2.getAbsolutePath();
        incident.addAttachment(path1);
        incident.addAttachment(path2);
        tempFilePaths.add(path1);
        tempFilePaths.add(path2);
        assertTrue(tempFile1.exists());
        assertTrue(tempFile2.exists());
  FileManager.deleteIncidentAttachments(incident);
        assertFalse(new File(path1).exists());
        assertFalse(new File(path2).exists());
    } @Test
    public void
testDeleteIncidentAttachments_Negative_FileDoes
NotExist() {   String nonExistentPath =
"non_existent_file_123456.txt";
        incident.addAttachment(nonExistentPath);
  FileManager.deleteIncidentAttachments(incident);
        assertTrue("Attachment list should still
contain the attachment",
incident.getAttachments().contains(nonExistentPat
h));
    }}
```

Test Case 16
Test Definition
Tests deletion of an incident when the user confirms the deletion and cancellation of incident deletion when user declines confirmation.

Input Value	
Incident with ID "1", user "user1", title "Test Incident", description "Desc" 1)Y 2)N	
Expected Value	**Actual Value**
1) Incident deleted successfully! 2) Deletion cancelled.	1)assertTrue(output.contains("v3.Incident deleted successfully!")) 2)assertTrue(output.contains("Deletion cancelled."))
Result of Test Case	*Successful*
Test Script	

```
public class DeleteIncidentTest {
   private final ByteArrayOutputStream
outContent = new
ByteArrayOutputStream();
   private final PrintStream originalOut =
System.out;
   private final InputStream originalIn =
System.in;
   private IncidentManager manager;
   private Map<String, List<Incident>>
userIncidents;
   @Before
   public void setUp() {
      System.setOut(new
PrintStream(outContent));
      userIncidents = new HashMap<>();
      userIncidents.put("user1", new
ArrayList<>());
 manager = new
IncidentManager(userIncidents, "user1");
   } @After
   public void tearDown() {
      System.setOut(originalOut);
      System.setIn(originalIn);
      outContent.reset();
   }@Test
   public void
testDeleteIncidentConfirmed() {
      Incident incident = new Incident("1",
"user1", "Test Incident", LocalDate.now(),
```

```
null, "Desc", "Open", "Low");
userIncidents.get("user1").add(incident);
        String input = "1\nY\n";
        System.setIn(new
ByteArrayInputStream(input.getBytes()));
        InputUtility.resetScanner();
        manager.deleteIncident();
        String output = outContent.toString();
  assertTrue(output.contains("v3.Incident
deleted successfully!"));
        assertEquals(0,
userIncidents.get("user1").size());
    }@Test
    public void
testDeleteIncidentCancelled() {
        Incident incident = new Incident("2",
"user1", "Cancel Incident",
LocalDate.now(), null, "Desc", "Open",
"Low");
userIncidents.get("user1").add(incident);
        String input = "1\nN\n";
        System.setIn(new
ByteArrayInputStream(input.getBytes()));
        InputUtility.resetScanner();
        manager.deleteIncident();
        String output = outContent.toString();
      assertTrue(output.contains("Deletion
cancelled."));
assertEquals(1,
userIncidents.get("user1").size());
    }}
```

Test Case 17	
Test Definition	
Tests editing an incident's title successfully.	
Input Value	
1) 1\n1\nNew Title\n8\n 2) Empty user incident list	
Expected Value	**Actual Value**
1) Incident updated successfully! 2) You have no incidents to edit.	1)assertTrue(output.contains("v3.Incident updated successfully!"))

	2)assertTrue(output.contains("You have no incidents to edit."))
Result of Test Case	***Successful***
Test Script	

```
public class EditIncidentTest {
   private final ByteArrayOutputStream
outContent = new
ByteArrayOutputStream();
   private final PrintStream originalOut =
System.out;
   private final InputStream originalIn =
System.in;
   private IncidentManager manager;
   private Map<String, List<Incident>>
userIncidents;
   @Before
   public void setUp() {
      System.setOut(new
PrintStream(outContent));
      userIncidents = new HashMap<>();
      userIncidents.put("user1", new
ArrayList<>());
      manager = new
IncidentManager(userIncidents, "user1");
   } @After
   public void tearDown() {
      System.setOut(originalOut);
      System.setIn(originalIn);
      outContent.reset();
  }  @Test
   public void testEditIncidentPositive() {
      Incident incident = new Incident("1",
"user1", "Old Title", LocalDate.now(), null,
"Old Description", "Open", "Low");
   userIncidents.get("user1").add(incident);
      String input = "1\n1\nNew Title\n8\n";
      System.setIn(new
ByteArrayInputStream(input.getBytes()));
      InputUtility.resetScanner();
      manager.editIncident();
      String output = outContent.toString();
```

```
  assertTrue(output.contains("v3.Incident
updated successfully!"));
      assertEquals("New Title",
incident.getTitle());
   }@Test
   public void testEditIncidentNegative() {
      String input = "";
      System.setIn(new
ByteArrayInputStream(input.getBytes()));
      InputUtility.resetScanner();
      manager.editIncident();
      String output = outContent.toString();
      assertTrue(output.contains("You
have no incidents to edit."));
   }}
```

Test Case 18	
Test Definition	
Tests that the getValidIntegerInput function correctly accepts valid input within the specified range, rejects input outside the range, and accepts subsequent valid input.	
Input Value	
1) "5\n" 2) "15\n7\n"	
Expected Value	**Actual Value**
1) Method should return 5 2) Method should reject 15 (out of range) and return 7 (valid)	1) assertEquals("Method should return the valid integer input", 5, result) 2) Result verified with assertEquals("Method should reject the out-of-range input and accept the subsequent valid input", 7, result)
Result of Test Case	***Successful***
Test Script	

```
public class GetValidIntegerInputTest {
   private final InputStream originalIn = System.in;
   @Before
   public void setUp() {
```

```
        InputUtility.resetScanner();
    }
    @After
    public void tearDown() {
        System.setIn(originalIn);
        InputUtility.resetScanner();
    }   @Test
    public void
testGetValidIntegerInput_PositiveCase() {
        ByteArrayInputStream in = new
ByteArrayInputStream("5\n".getBytes());
        System.setIn(in);
        InputUtility.resetScanner();
    int result = InputUtility.getValidIntegerInput(1, 10);
        assertEquals("Method should return the valid
integer input", 5, result);
    }
    @Test
    public void
testGetValidIntegerInput_NegativeCase() {
        ByteArrayInputStream in = new
ByteArrayInputStream("15\n7\n".getBytes());
        System.setIn(in);
      InputUtility.resetScanner();
   int result = InputUtility.getValidIntegerInput(1, 10);
        assertEquals("Method should reject the out-of-
range input and accept the subsequent valid input",
7, result);
    }
}
```

Test Case 19	
Test Definition	
Tests that the Incident ID is correctly set in the constructor.	
Input Value	
1) ID: "incident123" 2) ID: null	
Expected Value	**Actual Value**
1) incident.getId() should return "incident123" 2) NullPointerException should be thrown	1) Verified using assertEquals: expectedId equals incident.getId()

	2) Verified using @Test(expected = NullPointerException.class) annotation
Result of Test Case	***Successful***
Test Script	

```
public class IncidentIdTest {
   @Test
   public void testIncidentId_ShouldBeSetCorrectly()
{
  String expectedId = "incident123";
      Incident incident = new Incident(expectedId,
"user1", "Test Title", LocalDate.now(), null,
                                "Test description", "Open",
"High");
      assertEquals("The Incident ID should be set
correctly", expectedId, incident.getId());
   }
   @Test(expected = NullPointerException.class)
   public void
testIncidentId_ShouldThrowExceptionWhenIdIsNull(
) {
      new Incident(null, "user1", "Test Title",
LocalDate.now(), null, "Test description", "Open",
"High");
   }}
```

Test Case 20	
Test Definition	
Tests that the toString method includes all required fields and attachments and properly handles optional fields when they are null or empty.	
Input Value	
1) ID: "ID123" User: "eray" Title: "Test Title" Start Date: 2025-05-01 End Date: 2025-05-10 Description: "Detailed description" Status: "In Progress" Priority: "Medium" **Attachment: "incident_documents/ID123_file.txt"** **2) ID: "ID456" User: "user2" Title: "Missing Fields" Start Date: LocalDate.now() End Date: null Description: "Test" Status: "Started" Priority: "Low" No attachments added.**	
Expected Value	**Actual Value**

1)Incident ID "ID123" Description "Detailed description" Attachment path "incident_documents/ID123_file.txt" 2) Output string should contain "Not specified" for null end date Output string should NOT contain "Attached Documents" section	1) assertTrue on output.contains() for each expected element 2) assertTrue(output.contains("Not specified")) for null end date assertFalse(output.contains("Attached Documents")) for no attachments
Result of Test Case	***Successful***
Test Script	

```
public class IncidentToStringTest {
   @Test
   public void testToString_Positive() {
      Incident incident = new Incident("ID123",
"eray", "Test Title",
            LocalDate.of(2025, 5, 1),
LocalDate.of(2025, 5, 10),
            "Detailed description", "In
Progress", "Medium");
incident.addAttachment("incident_documents
/ID123_file.txt");
      String output = incident.toString();
      assertTrue(output.contains("ID123"));
      assertTrue(output.contains("Detailed
description"));
assertTrue(output.contains("incident_docume
nts/ID123_file.txt"));
   }
   @Test
   public void
testToString_Negative_MissingOptionalFields
() {
      Incident incident = new Incident("ID456",
"user2", "Missing Fields",
            LocalDate.now(), null,
            "Test", "Started", "Low");
      String output = incident.toString();
      assertTrue(output.contains("Not
specified")); // endDate null
      assertFalse(output.contains("Attached
Documents")); // no attachments
```

```
}}
```

Test Case 21

Test Definition

Tests that the initializeUsers method correctly adds predefined users to the users list, initializes an empty incident list for each user in the userIncidents map, and throws a NullPointerException when the users parameter is null.

Input Value

1) **Empty ArrayList for users 2) null for users parameter**

Expected Value	Actual Value
1) users list should contain 4 users 2) NullPointerException should be thrown with message "Users or UserIncidents cannot be null"	1)users list contains 4 users 2)NullPointerException is thrown with message "Users or UserIncidents cannot be null"

Result of Test Case	*Successful*

Test Script

```
public class InitializeUsersTest {
   private List<User> users;
   private Map<String, List<v3.Incident>>
userIncidents;
   private class TestInitializer {
      public void initializeUsers(List<User>
users, Map<String, List<v3.Incident>>
userIncidents) {
         if (users == null || userIncidents ==
null) {
            throw new
NullPointerException("Users or UserIncidents
cannot be null");
         }
         users.add(new User("oyku", "1234"));
         users.add(new User("eray", "1234"));
         users.add(new User("berkay", "1234"));
         users.add(new User("mehmet",
"1234"));
         for (User user : users) {
```

```
            userIncidents.put(user.getUserId(),
new ArrayList<>());
        }
        FileManager.loadIncidents(users,
userIncidents);
    } }
  @Before
  public void setUp() {
    users = new ArrayList<>();
    userIncidents = new HashMap<>();
  }@Test
  public void
testInitializeUsers_ShouldPopulateUsersAnd
Map() {
    TestInitializer initializer = new
TestInitializer();
    initializer.initializeUsers(users,
userIncidents);
    assertEquals(4, users.size());
assertTrue(userIncidents.containsKey("oyku"
));
assertTrue(userIncidents.get("mehmet").isEm
pty());
  }
  @Test(expected =
NullPointerException.class)
  public void
testInitializeUsers_ShouldThrowException_W
henUsersIsNull() {
    TestInitializer initializer = new
TestInitializer();
    initializer.initializeUsers(null,
userIncidents);
  }
}
```

Test Case 22
Test Definition
Tests that the getLineInput method correctly processes user input by trimming whitespace from the beginning and end of the input string.
Input Value

1)test input \n". 2)" \n"	
Expected Value	**Actual Value**
1) "test input" 2) ""	1) "test input" 2) ""
Result of Test Case	***successful***
Test Script	

```
public class InputHelperTest {
   private final InputStream systemIn =
System.in;
   private final PrintStream systemOut =
System.out;
   private ByteArrayInputStream testIn;
   private ByteArrayOutputStream
testOut;
   @Before
   public void setUpStreams() {
      testOut = new
ByteArrayOutputStream();
      System.setOut(new
PrintStream(testOut));
   }
   @After
   public void restoreSystemIO() {
      System.setIn(systemIn);
      System.setOut(systemOut);
   }
   private void provideInput(String data)
{
      testIn = new
ByteArrayInputStream(data.getBytes());
      System.setIn(testIn);
      InputUtility.resetScanner();
   }
   @Test
   public void
testGetLineInput_Positive() {
      provideInput("   test input   \n");
      String result =
InputUtility.getLineInput("Enter
something: ");
      assertEquals("test input", result);
```

```
    }
    @Test
    public void
testGetLineInput_Negative() {
        provideInput("     \n");
        String result =
InputUtility.getLineInput("Prompt: ");
        assertEquals("", result); }
}
```

Test Case 23

Test Definition

This test verifies that searching for incidents by a specific date correctly displays matching incidents if found, or a 'no incidents found' message if none match.

Input Value

1) "2025-05-13" 2) "2025-01-01"

Expected Value	Actual Value
The system should display a list of incidents matching the search date	The system displayed the incident matching the search date
Result of Test Case	***Successful***

Test Script

```
public class ListIncidentsByDateTest {
    private final ByteArrayOutputStream
outContent = new
ByteArrayOutputStream();
    private final PrintStream originalOut =
System.out;
    private final InputStream originalIn =
System.in;
    private IncidentManager manager;
    private Map<String, List<Incident>>
userIncidents;
    @Before
    public void setUp() {
        System.setOut(new
PrintStream(outContent));
        userIncidents = new HashMap<>();
        userIncidents.put("user1", new
ArrayList<>());
```

```
        manager = new
IncidentManager(userIncidents,
"user1");
    }@After
    public void tearDown() {
        System.setOut(originalOut);
        System.setIn(originalIn);
        outContent.reset();
    }@Test
    public void
testListIncidentsByDateWithMatch() {
        LocalDate date = LocalDate.of(2025,
5, 13);
        Incident incident = new
Incident("3", "user1", "Power Outage",
date, date, "Power down.", "Open",
"Medium");
userIncidents.get("user1").add(incident);
        String input = "2025-05-13\n0\n";
        ByteArrayInputStream in = new
ByteArrayInputStream(input.getBytes());
        System.setIn(in);
        InputUtility.resetScanner();
        manager.listIncidentsByDate();
        String output =
outContent.toString();
        assertTrue(output.contains("Power
Outage"));
    assertTrue(output.contains("Incidents
for 2025-05-13"));
    }@Test
    public void
testListIncidentsByDateWithNoMatch() {
        LocalDate date = LocalDate.of(2024,
12, 25);
   Incident incident = new Incident("4",
"user1", "Server Down", date, date,
"Server issues.", "Closed", "High");
userIncidents.get("user1").add(incident);
        String input = "2025-01-01\n";
        ByteArrayInputStream in = new
ByteArrayInputStream(input.getBytes());
        System.setIn(in);
```

```
        InputUtility.resetScanner();
       manager.listIncidentsByDate();
    String output = outContent.toString();
        assertTrue(output.contains("No
incidents found for the date: 2025-01-
01"));
    }
}
```

Test Case 24	
Test Definition	
This test verifies that the loadIncidents() method correctly loads incidents from an existing file or handles a missing file gracefully by leaving the incident list empty.	
Input Value	
1) ID "eray" and password "1234""eray_incident.txt" 2)ID "yok" and password "yok"No file named	
Expected Value	**Actual Value**
1) load incidents from the "eray_incident.txt" file 2) load incidents but find no file	Same with the expected
Result of Test Case	*successful*
Test Script	

```
public class LoadIncidentsTest {
    @Test
    public void testLoadIncidents_Positive_FileExists() {
        List<User> users = new ArrayList<>();
        users.add(new User("eray", "1234"));
        Map<String, List<Incident>> incidentMap = new
HashMap<>();
        incidentMap.put("eray", new ArrayList<>());
        FileManager.loadIncidents(users, incidentMap);
        assertNotNull(incidentMap.get("eray"));
    }
@Test
    public void
testLoadIncidents_Negative_FileDoesNotExist() {
        List<User> users = new ArrayList<>();
        users.add(new User("ghost", "nopass"));
        Map<String, List<Incident>> map = new HashMap<>();
```

```
        map.put("ghost", new ArrayList<>());
        FileManager.loadIncidents(users, map);
        assertTrue("Should remain empty if no file exists",
map.get("ghost").isEmpty());
    }
}
```

Test Case 25

Test Definition

This test verifies that the manage attachments functionality correctly adds new attachments, handles invalid inputs, and manages cases where no attachments exist for removal.

Input Value

1) "1" (Select "Add new attachment" option)
Path to the temporary file (Adding the temporary file as an attachment)
"3" (Select "Return to edit menu" option to exit)
2) "5" (Invalid menu option - should be handled)
"2" (Select "Remove attachment" option)
"0" (Cancel removal when prompted)
"3" (Select "Return to edit menu" option to exit)

Expected Value	Actual Value
1) "Manage Attachments" indicating the menu 2) invalid input	1)"Manage Attachments" menu 2)invalid input
Result of Test Case	***successful***

Test Script

```
public class ManageAttachmentsTest {
    private final ByteArrayOutputStream outContent = new
ByteArrayOutputStream();
    private final PrintStream originalOut = System.out;
    private final InputStream originalIn = System.in;
    private Incident incident;
    private IncidentManager manager;
    private File tempFile;
    @Before
    public void setUp() throws IOException {
        System.setOut(new PrintStream(outContent));
```

```
        new File("documents/").mkdirs();
        tempFile = File.createTempFile("testDoc", ".txt");
        tempFile.deleteOnExit();
        incident = new Incident("INC001","TestUser","Test
Incident",LocalDate.now(),null,"Test description","Open",
"High"  );
        manager = new IncidentManager(new HashMap<>(),
"testUser");
        InputUtility.resetScanner();
    }
    @After
    public void tearDown() {
        System.setOut(originalOut);
        System.setIn(originalIn);
        InputUtility.resetScanner();
    } @Test
    public void testManageAttachmentsPositive() {
        String input = String.format("1\n%s\n3\n",
tempFile.getAbsolutePath());
        System.setIn(new
ByteArrayInputStream(input.getBytes()));
        InputUtility.resetScanner();
        manager.manageAttachments(incident);
        boolean matched = incident.getAttachments().stream()
                .anyMatch(path ->
path.contains(tempFile.getName()));
        System.out.println("Expected: " + tempFile.getName());
        incident.getAttachments().forEach(System.out::println);
        assertTrue("Attachment was not added correctly",
matched);
        assertTrue(outContent.toString().contains("Manage
Attachments"));
    }@Test
    public void
testManageAttachmentsNegative_InvalidAndCancelRemove()
{  String input = "5\n2\n0\n3\n";  // invalid option, try to
remove when none, then exit
        System.setIn(new
ByteArrayInputStream(input.getBytes()));
        InputUtility.resetScanner();
        manager.manageAttachments(incident);
        String output = outContent.toString();
```

```
        assertTrue(output.contains("Manage Attachments"));
        assertTrue(output.contains("No attachments to
remove"));
    }
```

Test Case 26	
Test Definition	
Tests that the parseIncident method correctly parses a valid incident string and adds it to the userIncidents map and handles invalid date formats gracefully without adding malformed incidents.	
Input Value	
1) userId: "testUser" validIncident: A properly formatted incident string containing ID, title, dates, description, status, priority, and attachments 2) userId: "testUser" malformedIncident: An incident string containing an invalid date format ("INVALID_DATE")	
Expected Value	**Actual Value**
1) userIncidents.get(userId).size() should be 1 2) userIncidents.get(userId) should be empty	1)userIncidents.get(userId).size() is 1 2)userIncidents.get(userId) is empty
Result of Test Case	*successful*
Test Script	

```
public class ParseIncidentTest {
    private Map<String, List<Incident>>
userIncidents;
    private String userId;
    @Before
    public void setUp() {
        userId = "testUser";
        userIncidents = new HashMap<>();
        userIncidents.put(userId, new ArrayList<>());
    }@Test
    public void testParseIncident_Positive() {
        String validIncident ="===============\n" +
                "v3.Incident ID: 12345\n" +
                "Title: Sample Incident\n" +
```

```
            "Start Date: 2024-01-10\n" +
            "End Date: 2024-01-12\n" +
            "Description: Something went wrong.\n" +
            "Status: Open\n" +
            "Priority: High\n" +
            "- doc1.txt\n" +
            "- doc2.pdf\n" +
            "==========================\n";
        FileManager.parseIncident(validIncident,
userId, userIncidents);
        assertEquals(1,
userIncidents.get(userId).size());
    }@Test
    public void
testParseIncident_Negative_InvalidDateFormat() {
        String malformedIncident ="============\n"
+
            "v3.Incident ID: 999\n" +
            "Title: Invalid Date\n" +
            "Start Date: INVALID_DATE\n" +
            "Description: This should fail
gracefully.\n" +
            "Status: Closed\n" +
            "Priority: Medium\n" +
            "==========================\n";
        FileManager.parseIncident(malformedIncident,
userId, userIncidents);
      assertTrue(userIncidents.get(userId).isEmpty());
    }
```

Test Case 27	
Test Definition	
The test verifies that the removeDocument method successfully removes an existing document file and updates the incident's attachments list accordingly, and also checks its behavior when attempting to remove a file that doesn't exist in the filesystem.	
Input Value	
1) testFilePath 2) nonExistentPath	
Expected Value	**Actual Value**

1) The result of removeDocument should be true 2) The result of removeDocument should be false	1)removeDocument is true 2)removeDocument is false
Result of Test Case	***successful***
Test Script	

```
public class RemoveDocumentTest {
   private Incident incident;
   private String testFilePath;
   @Before
   public void setUp() throws IOException {
      incident = new
Incident(UUID.randomUUID().toString().substring(0,
8), "testUser",
            "Test Title", null, null, "Test Description",
"Open", "Low");
      testFilePath = "removable_test_doc.txt";
      File file = new File(testFilePath);
      if (!file.exists()) {
         try (FileWriter writer = new FileWriter(file)) {
            writer.write("Temporary test file content.");
         } }
      incident.addAttachment(testFilePath);
   }@After
   public void tearDown() {
      File file = new File(testFilePath);
      if (file.exists()) file.delete();
   } @Test
   public void testRemoveDocument_Positive() {
      assertTrue(new File(testFilePath).exists());
      boolean result =
FileManager.removeDocument(incident,
testFilePath);
assertTrue(result);assertFalse(new
File(testFilePath).exists());
assertFalse(incident.getAttachments().contains(test
FilePath));
   }@Test
   public void
testRemoveDocument_Negative_FileDoesNotExist()
{  String nonExistentPath =
"this_file_does_not_exist.txt";
```

```
boolean result=
FileManager.removeDocument(incident,
nonExistentPath);
        assertFalse(result);
assertFalse(incident.getAttachments().contains(non
ExistentPath));
    }
}
```

Test Case 28	
Test Definition	
<Describe the test briefly>	
Input Value	
<Write input>	
Expected Value	**Actual Value**
<Write expected result>	<Write actual result>
Result of Test Case	*successful*
Test Script	

```
public class SaveAllIncidentsTest {
    private static final String TEST_USER =
"testUser";
    private static final String FILE_NAME =
TEST_USER + "_incident.txt";
    @Before
    public void setUp() throws IOException {
     Files.deleteIfExists(Paths.get(FILE_NAME));
    }
    @After
    public void tearDown() throws IOException {
     Files.deleteIfExists(Paths.get(FILE_NAME));
    }
    @Test
    public void testSaveAllIncidents_Positive()
throws IOException {
        Map<String, List<Incident>>
userIncidents = new HashMap<>();
        List<Incident> incidents = new
ArrayList<>();
```

```
        Incident incident = new Incident(
                "1234", TEST_USER, "Sample
Incident",
                LocalDate.of(2024, 5, 1),
LocalDate.of(2024, 5, 2),
                "Description here", "Open", "High"
        );
        incidents.add(incident);
        userIncidents.put(TEST_USER,
incidents);
FileManager.saveAllIncidents(userIncidents);
        Path path = Paths.get(FILE_NAME);
        assertTrue("File should be created",
Files.exists(path));
        String content = new
String(Files.readAllBytes(path));
        assertTrue("File should contain incident
title", content.contains("Sample Incident"));
    }
    @Test
    public void
testSaveAllIncidents_NegativeWithInvalidFile()
{
        Map<String, List<Incident>>
userIncidents = new HashMap<>();
        List<Incident> incidents = new
ArrayList<>();

        Incident incident = new Incident(
                "9999", "/invalidUser", "Bad Path
Test",
                LocalDate.now(), null, "Invalid write
test", "Open", "Low"
        );
        incidents.add(incident);
        userIncidents.put("/invalidUser",
incidents);
FileManager.saveAllIncidents(userIncidents);
        Path invalidPath =
Paths.get("/invalidUser_incident.txt");
        assertFalse("Invalid file should not be
created", Files.exists(invalidPath));
    }
```

```
}
```

Test Case 29

Test Definition

The test verifies that the saveAllIncidents method successfully writes incident data to a file when provided with valid user incidents, and also tests its behavior when attempting to write to an invalid file path.

Input Value

1) userIncidents: A map containing a test user with one incident TEST_USER: "testUser" incident: A sample incident with ID "1234", specified dates, description, status, and priority 2) userIncidents: A map containing an invalid user path with one incident incident: A sample incident with an invalid user path "/invalidUser"

Expected Value	Actual Value
1) "testUser_incident.txt" should be created 2) "/invalidUser_incident.txt" should not be created	1) "testUser_incident.txt" is created 2) "/invalidUser_incident.txt" is not created

Result of Test Case	*successful*

Test Script

```
public class ViewIncidentTest {
   private final ByteArrayOutputStream outContent = new ByteArrayOutputStream();
   private final PrintStream originalOut = System.out;
   private final InputStream originalIn = System.in;
   private IncidentManager manager;
   private Map<String, List<Incident>> userIncidents;
   @Before
   public void setUp() {
      System.setOut(new PrintStream(outContent));
      userIncidents = new HashMap<>();
      userIncidents.put("user1", new ArrayList<>());
      manager = new IncidentManager(userIncidents, "user1");
   }
   @After
   public void tearDown() {
```

```
        System.setOut(originalOut);
        System.setIn(originalIn);
        outContent.reset();
    }
    @Test
    public void testViewIncidentsPositive() {
        Incident incident = new Incident("1",
"user1", "Power Outage", LocalDate.now(),
null, "No electricity", "Open", "High");
        userIncidents.get("user1").add(incident);
        String input = "1\n";
        System.setIn(new
ByteArrayInputStream(input.getBytes()));
        InputUtility.resetScanner();
        manager.viewIncidents();
        String output = outContent.toString();
        assertTrue(output.contains("=====  Your
Incidents =====\"));
        assertTrue(output.contains("Power
Outage"));
        assertTrue(output.contains("No
electricity"));
    }
    @Test
    public void testViewIncidentsEmptyList() {
        String input = "";
        System.setIn(new
ByteArrayInputStream(input.getBytes()));
        InputUtility.resetScanner();
        manager.viewIncidents();
        String output = outContent.toString();
        assertTrue(output.contains("You have no
incidents."));
    }
}
```

Test Case 30
Test Definition

The test verifies that the viewPreviousIncidents method displays an appropriate message when no incidents exist in the system, and correctly displays all incidents across all users when incidents are present.	
Input Value	
1) userIncidents: A map containing two users, each with one incident currentUser: "testUser" Input stream: "0\n" Two incidents: one for testUser and one for otherUser with different details 2) emptyUserIncidents: A map containing one user with an empty incident list currentUser: "testUser"	
Expected Value	**Actual Value**
1) ===== View Previous Incident Posts ===== Test Incident" Other Incident testUser otherUser 2) No incidents have been recorded in the system	Same with the expected values.
Result of Test Case	***successful***
Test Script	

```
public class ViewPreviousIncidentsTest {
    private IncidentManager incidentManager;
    private Map<String, List<Incident>>
userIncidents;
    private final String currentUser = "testUser";
    private final ByteArrayOutputStream
outputStream = new ByteArrayOutputStream();
    private final PrintStream originalOut =
System.out;
    @Before
    public void setUp() {
        System.setOut(new
PrintStream(outputStream));
        userIncidents = new HashMap<>();
        List<Incident> incidents = new ArrayList<>();
        userIncidents.put(currentUser, incidents);
        userIncidents.put("otherUser", new
ArrayList<>());
        Incident testIncident = new Incident(
                "12345678",
```

```
            currentUser,
            "Test Incident",
            LocalDate.now(),
            LocalDate.now().plusDays(5),
            "Test description",
            "Open",
            "High"
        );
        incidents.add(testIncident);
        Incident otherIncident = new Incident(
            "87654321",
            "otherUser",
            "Other Incident",
            LocalDate.now().minusDays(2),
            LocalDate.now().plusDays(3),
            "Other description",
            "In Progress",
            "Medium"
        );
userIncidents.get("otherUser").add(otherIncident);
        incidentManager = new
IncidentManager(userIncidents, currentUser);
    }
    @After
    public void tearDown() {
        System.setOut(originalOut);
    }
    @Test
    public void
testViewPreviousIncidents_WithIncidents() {
        ByteArrayInputStream inputStream = new
ByteArrayInputStream("0\n".getBytes());
        System.setIn(inputStream);
        incidentManager.viewPreviousIncidents();
        String output = outputStream.toString();

        assertTrue(output.contains("===== View
Previous v3.Incident Posts =====));
        assertTrue(output.contains("Test Incident"));
        assertTrue(output.contains("Other
Incident"));
```

```
        assertTrue(output.contains("testUser"));
        assertTrue(output.contains("otherUser"));
    }
    @Test
    public void
testViewPreviousIncidents_NoIncidents() {
        Map<String, List<Incident>>
emptyUserIncidents = new HashMap<>();
        emptyUserIncidents.put(currentUser, new
ArrayList<>());
        IncidentManager emptyIncidentManager =
new IncidentManager(emptyUserIncidents,
currentUser);
        outputStream.reset();
   emptyIncidentManager.viewPreviousIncidents();
        String output = outputStream.toString();
        assertTrue(output.contains("No incidents
have been recorded in the system."));
    }
}
```

4. CONCLUSION

This project presents a structured and evolving Incident Management System developed in Java, successfully implemented across three progressive versions. In Version 1, high-priority functional requirements such as user authentication, incident creation, and the assignment of status and priority were fully implemented, establishing the system's core functionality. With Version 2, moderate-priority features—such as setting incident dates, editing/deleting reports, and document uploads—added flexibility and depth to the user experience. Finally, Version 3 introduced user-friendly enhancements like filtering incidents by date and reviewing historical reports, making the system more effective for long-term incident tracking and reference.

The accompanying Test Case Document ensured that each feature was rigorously tested for functionality, usability, and reliability. Through detailed test scenarios, it confirmed that all components worked as intended, supported regression testing, and upheld system integrity following updates.